

Bài tập thực hành tuần 9

Họ và tên: Nguyễn Văn Phúc Huy

MSSV: 23110163

Lớp: 23TTH (Sáng thứ 6)

```
In [17]: import math
import matplotlib.pyplot as plt
import numpy as np
import time
import random
import heapq
import sys
from time import perf_counter
from math import log2
```

Bài tập:

Câu hỏi:

Ứng với số tự nhiên n , ta gọi $f(n)$ là tổng bình phương tất cả các chữ số của n như sau:

Cho trước số tự nhiên N bất kỳ, ta xây dựng dãy số sau đây:

$$a_{n+1} = f(a_n) \text{ và } a_0 = N.$$

Viết thuật toán tìm chu kỳ tuần hoàn nhỏ nhất của dãy số đó với giá trị N cho trước.

Ví dụ: nếu $N = 85$, dãy sẽ có giá trị lần lượt như sau:

$$85 \rightarrow 89 \rightarrow 145 \rightarrow 42 \rightarrow 20 \rightarrow 4 \rightarrow 16 \rightarrow 37 \rightarrow 58 \rightarrow 89 \rightarrow 145 \rightarrow \dots$$

```
def f(n):
    result = 0
    while n > 0:
        result += (n % 10) ** 2
        n //= 10
    return result
```

```
def find_cycle(n):
```

```

seq = []
last_number = n

print(n, end=' -> ')
while last_number not in seq:
    seq.append(last_number)
    new_number = f(last_number)
    if new_number in seq:
        print(new_number)
        seq.append(new_number)
        break

    print(new_number, end=' -> ')
    last_number = new_number

end_value = seq[-1]
i = len(seq) - 2
while seq[i] != end_value:
    i -= 1

print(f'Smallest cycle: ', end='')
while i < len(seq):
    print(seq[i], end=' ')
    i += 1

n = 85
print(f'{n = }')
find_cycle(n)

print()

n = 23110163
print(f'{n = }')
find_cycle(n)

```

Output:

```

n = 85
85 -> 89 -> 145 -> 42 -> 20 -> 4 -> 16 -> 37 -> 58 -> 89
Smallest cycle: 89 145 42 20 4 16 37 58 89
n = 23110163
23110163 -> 61 -> 37 -> 58 -> 89 -> 145 -> 42 -> 20 -> 4 -> 16 -> 37
Smallest cycle: 37 58 89 145 42 20 4 16 37

```

Hàm `f(n)` tính tổng bình phương của các chữ số của n.

Hàm `find_cycle(n)` tìm chu kỳ tuần hoàn nhỏ nhất của dãy số được tạo ra bởi việc liên tục áp dụng hàm f lên n. Hoạt động của hàm này như sau:

1. Tạo một danh sách `seq` để lưu trữ các số đã xuất hiện trong quá trình tính toán.
2. Sử dụng một vòng lặp để tiếp tục tính toán cho đến khi một số lặp lại xuất hiện trong `seq`.

3. Trong mỗi vòng lặp, tính số mới bằng cách áp dụng hàm f lên số cuối cùng và kiểm tra xem nó đã xuất hiện trong `seq` chưa. Nếu đã xuất hiện, in ra số đó và dừng vòng lặp.
4. Sau khi tìm được số lặp lại, xác định chu kỳ tuần hoàn nhỏ nhất bằng cách tìm vị trí của số lặp lại trong `seq` và in ra các số từ vị trí đó đến cuối `seq` như là chu kỳ tuần hoàn nhỏ nhất.

Xét về độ phức tạp thời gian, hàm $f(n)$ có độ phức tạp $O(d)$ với d là số chữ số của n .

Hàm `find_cycle(n)` có độ phức tạp $O(m)$ với m là độ dài của chuỗi số trước khi lặp lại xuất hiện bởi vì trong trường hợp xấu nhất, tất cả các số đều khác nhau cho đến khi một số lặp lại xuất hiện. Tuy nhiên, do tính chất của hàm f , số lượng các số có thể xuất hiện sẽ bị giới hạn, nên độ phức tạp thực tế có thể được coi là $O(1)$ trong trường hợp này.

Như vậy, tổng thể độ phức tạp thời gian của chương trình là $O(d) + O(m)$ với d và m đều là các hằng số, nên thuật toán này có thể được coi là $O(1)$ trong thực tế do giới hạn của các số có thể xuất hiện.